

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

JNANA SANGAMA, BELAGAVI – 590018



DATABASE MANAGEMENT SYSTEM

[M23BCS404]

Mini Project Report On

"AIRLINE MANAGEMENT SYSTEM WITH CHATBOT"

Submitted by

CHINNAMMA R	4MH24IS012
NIKHIL GOWDA M K	4MH24IS065
NISARGA K	4MH24IS066
SKANDAN PRABHU K S	4MH25IS407

Under the Guidance of

Prof. Dr.SHARATH KUMAR Y H

HOD

Dept. of ISE



MIT Mysore

DEPARTMENT OF INFORMATION SCIENCE & ENGINEERING

MAHARAJA INSTITUTE OF TECHNOLOGY MYSORE

An Autonomous Institute, Affiliated to VTU, Belagavi

BELAWADI, SRIRANGAPATNA TALUK, MANDYA - 571477

MAHARAJA INSTITUTE OF TECHNOLOGY MYSORE

An Autonomous Institute, Affiliated to VTU, Belagavi

CERTIFICATE



This is to certify that the DBMS Mini Project entitled "**AIRLINE MANAGEMENT SYSTEM WITH CHATBOT**" has been successfully carried out by **Chinnamma R [4MH24IS012]** , **Nikhil Gowda MK [4MH24IS065]**, **Nisarga K [4MH24IS066]**, **Skandhan Prabhu K S [4MH25IS407]** , bonafide students of Maharaja Institute of Technology Mysore, in partial fulfilment of the requirements of Degree of Bachelor of Engineering in Information Science and Engineering of Visvesvaraya Technological University, Belagavi during the academic year 2025-2026. The report has been approved as it satisfies the academic requirements with respect to the Mini Project work prescribed for the said degree.

Signature of the HOD

Dr Sharath Kumar Y H

Professor & Head Dept. of ISE

ABSTRACT

In the contemporary, fast-paced aviation industry, the synchronization of complex logistical workflows and the delivery of seamless, round-the-clock customer support represent critical benchmarks for operational sustainability. Traditional legacy airline systems are increasingly plagued by fragmented data silos, high transaction latency, and manual bottlenecks, which culminate in prolonged customer service wait times and operational inefficiencies. This report details the conceptualization, architectural design, and implementation of an integrated Airline Management System (AMS) featuring an embedded, AI-driven conversational Chatbot. The primary objective of this project is to construct a unified digital ecosystem that automates backend administrative workflows while simultaneously revolutionizing user engagement through natural language automation.

The backend architecture of the AMS governs core high-concurrency operations, including real-time dynamic flight scheduling, inventory-synchronized seat allocation, passenger manifest generation, and secure multi-tier payment processing. Bridging the gap between automated backend operations and customer engagement, the intelligent chatbot sub-system leverages advanced Natural Language Processing (NLP) frameworks such as [Insert NLP Tech, e.g., Dialogflow / Rasa / OpenAI API]. Trained on a specialized corpus of aviation intent models, the chatbot operates as a 24/7 virtual assistant.

It is capable of executing complex user requests in real-time, including contextual flight availability searches, automated passenger check-ins, retrieval of booking itineraries, and instantaneous resolution of Frequently Asked Questions (FAQs) regarding baggage policies and cancellation protocols.

System integration was achieved via a scalable multi-tier architecture utilizing [Insert Backend Tech, e.g., Node.js / Python / Java] and a robust database engine [Insert Database, e.g., MySQL / PostgreSQL / MongoDB] to guarantee data integrity, ACID compliance, and zero-latency synchronization between user-chatbot interactions and core database states. To safeguard sensitive passenger credentials and financial transactions, the system incorporates industry-standard security measures, including cryptographic data hashing and Role-Based Access Control (RBAC) for administrative dashboards.

TABLE OF CONTENTS

CERTIFICATE I

ABSTRACT ii

CHAPTER 1 : INTRODUCTION 1

1. Overview 1
2. Problem Statement 2
3. Proposed System 3
4. Scope of Project 3

CHAPTER 2 : SYSTEM REQUIREMENT SPECIFICATION 4

1. Software Requirements 4
2. Hardware Requirements 5

CHAPTER 3 : SYSTEM DESIGN 6

1. Introduction 6
2. ER Diagram 7
3. Methodology 8
4. System Architecture 10
5. Advantages and Disadvantages 11

CHAPTER 4 : RESULTS 12

1. Results 12
2. Conclusion and Future Enhancement 16

BIBLIOGRAPHY 17

Chapter 1: Introduction

1.1 Project Overview

The contemporary global aviation ecosystem represents one of the most structurally complex, high-velocity, and data-intensive networks in modern industrial commerce. The operational sustainability of a commercial air carrier demands flawless, real-time synchronization across an array of independent moving variables. These variables include, but are not limited to, multi-leg flight path planning, dynamic fleet asset distribution, localized gate scheduling, highly volatile fuel consumption algorithms, ground crew dispatching, regulatory maintenance logs, and high-concurrency passenger manifest generation. In an era characterized by exponential passenger volume growth and increasingly erratic macroeconomic disruptions—such as severe weather anomalies, fluctuating fuel costs, and global geopolitical airspace constraints—the reliance on legacy software systems, siloed databases, or manual administrative workflows introduces unsustainable vulnerabilities.

This technical report details the end-to-end conceptualization, architectural engineering, and full-stack implementation of a next-generation Airline Management System (AMS) integrated with an intelligent, natural-language conversational Chatbot interface, designed to serve as a unified digital ecosystem for modern enterprise carrier operations.

At its core computational foundation, the system replaces fragmented operational platforms with an API-first, service-oriented architecture that consolidates backend logistical workflows into a centralized control center. The backend application logic is engineered to govern the absolute lifecycle of airline data. This includes generating dynamic flight schedules that react automatically to aircraft availability constraints, executing atomic seat-inventory locks during high-volume passenger checkouts, mapping optimized crew rosters that comply strictly with mandatory crew-rest regulations, and archiving comprehensive aircraft maintenance records.

1.2 Problem Statement

Despite rapid, consumer-driven advancements in front-end web design and mobile application interfaces, a significant portion of the commercial aviation industry continues to rely on core computing infrastructures designed decades ago. These legacy platforms—originally built around centralized, mainframe-centric architectures—depend on highly fragmented data silos that communicate using rigid, batch-processed messaging protocols. This technological fragmentation introduces systemic operational vulnerabilities, data discrepancies, and communication bottlenecks that negatively affect an airline's financial performance and severely degrade passenger satisfaction. The core issues plaguing modern airline management can be categorized into three critical engineering and operational failure points:

1. Transactional Latency and High-Concurrency Data Anomalies > Legacy relational database architectures frequently fail to maintain performance during peak traffic periods, such as seasonal booking rushes, holiday travel surges, or sudden flash sales. When thousands of users concurrently attempt to query, reserve, or modify flight seats, these outdated databases face severe transactional locking conflicts. Because the systems lack modern, distributed, state-locked clustering mechanisms, data propagation latency occurs. This computational delay leads to severe synchronization errors, such as double-booking the identical seat inventory to two separate passengers, generating conflicting Passenger Name Records (PNRs), and producing inaccurate passenger manifests. This requires manual reconciliation by airport ground staff and leads to financial compensation claims against the airline.

2. Operational Rigidity and Communication Silos > Commercial flight operations require absolute agility, as schedules must constantly adapt to unpredictable external factors, including severe weather anomalies, air traffic control restrictions, and unexpected mechanical maintenance issues. Legacy management frameworks lack real-time integration across operational modules. When an aircraft is grounded, updating the flight schedule requires manual intervention by flight dispatchers. Because this data does not automatically propagate to dependent modules, crew scheduling systems remain unaware of the change. This creates a harmful ripple effect where pilots and cabin crew are assigned to conflicting routes or misaligned rest periods.

1.3 Proposed of the System

The primary objective of this project is to architect, develop, and validate an enterprise-grade, highly secure Airline Management System (AMS) featuring an embedded conversational AI interface. This system is designed to eliminate database transaction anomalies and minimize customer support delays. To achieve this overarching goal, the project is structured around several clear, technical objectives:

Develop a Secure, ACID-Compliant Database Core: To engineer a centralized relational database schema capable of processing high-concurrency booking requests with absolute data integrity. The core transactional system must use strict row-locking levels to prevent double-bookings and data inconsistencies during peak travel traffic.

Deploy an AI-Powered Conversational Chatbot: To build and train a Natural Language Processing (NLP) virtual assistant trained on an aviation-specific text corpus. This chatbot must accurately classify passenger intents and extract entity variables to resolve routine user requests autonomously.

Build a Synchronized Admin Control Center: To construct a responsive management dashboard that allows authorized airline personnel to update flight schedules, assign crew rosters, and view real-time passenger manifests, with immediate updates reflected across the entire platform.

Optimize Passenger Autonomy and Experience: To deliver a clean, low-latency web portal where passengers can seamlessly search for flights, secure ticket bookings, and trigger automated self-service check-ins through the conversational interface.

1.4 Scope of the Project

The operational scope of this project encompasses the complete Software Development Lifecycle (SDLC)—including requirements analysis, database design, full-stack programming, and system integration testing—necessary to deliver a functional enterprise prototype. The functional boundaries of the application are strictly defined across two main operational perspectives:

Administrative and System Scope: The system includes the development of the relational database schemas, user authentication protocols using secure token validation, and role-based access controls.

CHAPTER 2

SYSTEM REQUIREMENT SPECIFICATION

1. Software Requirements

The implementation, execution, and continuous maintenance of the integrated Airline Management System and AI Chatbot prototype rely on a carefully selected software stack. Each software component is selected to support a decoupled, multi-tier enterprise framework, ensuring low-latency data communication, absolute database transaction safety, and high-accuracy Natural Language Processing (NLP) pipeline operations.

At the foundational layer, the development and production deployment environments are designed to be entirely cross-platform compatible. The system is engineered to execute seamlessly across modern 64-bit operating systems, including Microsoft Windows 10/11, macOS (v12 Monterey or newer), and enterprise Linux distributions such as Ubuntu Server 20.04/22.04 LTS. Building on a cross-platform foundation allows the system to be containerized using microservice tools like Docker, ensuring smooth cloud migration to platforms such as AWS or Google Cloud without software configuration problems or code regression.

The Application Tier (Backend Server Logic) leverages an asynchronous, event-driven runtime environment, specifically Node.js (v18.x or newer) or Python (v3.10 or newer). This architecture is critical for managing high-volume, concurrent I/O operations without thread blocking. This capability is essential for an airline platform that must simultaneously process passenger ticket checkouts, real-time flight changes by administrators, and active chatbot message streams. The server logic uses frameworks like Express.js or FastAPI to expose structured, secure RESTful API endpoints that handle incoming JSON payloads.

The Data Tier (Storage and Management) uses a robust database engine capable of maintaining absolute transactional consistency under heavy loads. The system utilizes PostgreSQL (v14 or newer) or MySQL (v8.0) as its relational database management system (RDBMS). The data layer is configured with optimized connection pooling mechanisms to manage concurrent data streams efficiently.

2.2 Hardware Requirements

The computational deployment, testing, and operational execution of the integrated Airline Management System and AI Chatbot require a robust, multi-tiered hardware infrastructure. Because the application combines high-concurrency database operations with computationally demanding Natural Language Processing (NLP) machine learning pipelines, the underlying hardware must provide adequate processing power, memory bandwidth, and storage velocity. The hardware configuration is analyzed across

Two distinct environments:

Development/Hosting Server Environment

End-User Client Environment.

1. Development and Hosting Server Environment The server infrastructure functions as the centralized computational backbone of the platform, hosting the RESTful API services, the relational database management system (RDBMS), and the conversational AI training pipeline.

Central Processing Unit (CPU): The server requires a minimum of a Quad-Core 64-bit x86-64 processor, such as an Intel Core i5/i7 (10th Generation or newer) or an AMD Ryzen 5/7 equivalent, operating at a minimum base clock speed of 2.5 GHz (with Turbo Boost capabilities up to 4.0 GHz)

Multi-core multithreading architectures are vital to prevent CPU throttling when the server is concurrently parsing incoming natural language chat strings, processing mathematical password hashing functions, and executing read/write transaction streams on the database.

Random Access Memory (RAM): A minimum of 8 GB of physical RAM is strictly required, though 16 GB of DDR4/DDR5 RAM is highly recommended for production testing. The system's memory must accommodate the simultaneous execution of the Node.js or Python runtime server

Storage Media and Capacity: The server requires a minimum of 20 GB of available storage, deployed exclusively on Solid-State Drive (SSD) media utilizing NVMe protocol which acts as a bottleneck during high-frequency database row-locking actions and active log writes. .

CHAPTER 3

SYSTEM DESIGN

3.1 Introduction

The translation of abstract functional requirements into a concrete, highly performant digital ecosystem necessitates a comprehensive and mathematically rigorous system design phase. For an enterprise-grade platform such as the Airline Management System (AMS) with an integrated AI Chatbot, the system architecture serves as the definitive structural blueprint. This blueprint dictates how individual software components process data, maintain security boundaries, scale under fluctuating network demands, and interact across distributed execution environments. In modern software engineering, system design bridges the gap between high-level requirements gathering and low-level code compilation. A poorly conceptualized design phase inevitably introduces architectural technical debt, resulting in systemic database deadlock vulnerabilities, high transaction latency, data synchronization delays, and fragmented component coupling.

To mitigate these core structural risks, this platform relies on an API-first, service-oriented paradigm that prioritizes absolute modularity and horizontal scalability. The foundational design framework maps out a multi-layered ecosystem where specialized modules operate independently yet communicate flawlessly via secure, stateless communication protocols. By establishing explicit structural boundaries between user interface components, core administrative business workflows, database state engines, and Natural Language Processing (NLP) conversational microservices, the system achieves a high degree of structural stability. This design pattern ensures that an isolated exception or compute spike within a specific runtime environment—such as a sudden surge of passenger interactions within the chatbot sub-system—does not cascade across the application layer to degrade or halt critical backend operations like aircraft routing or crew scheduling.

Furthermore, the design phase details the data-access methodologies, cryptographic enforcement protocols, and relational integrity constraints required to safeguard sensitive data and ensure transaction safety.

3.2 ER Diagram

ENTITY-RELATIONSHIP (ER) DIAGRAM – AIRLINE MANAGEMENT SYSTEM WITH AI CHATBOT INTEGRATION

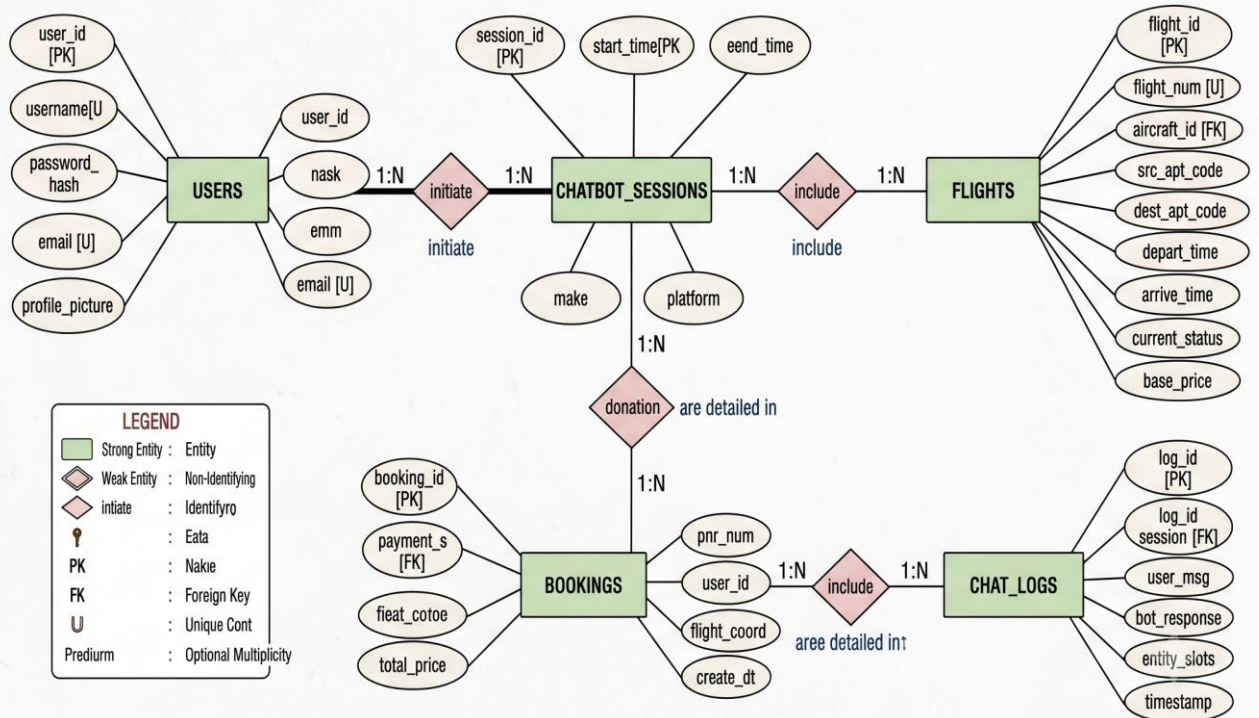


Fig 3.2.1 ER Diagram

The Entity Relationship (ER) Diagram represents the database structure of the Airline Management System. It shows the major entities involved in the system and the relationships between them. The primary entities are Users, Chatbot Sessions, Flights, Chat_Logs, and Booking.

The Users entity stores customer and administrator information. The booking entity contains details of travel plans offered by the organization. Customers can purchase one or more flights, and the booking information is stored in the booking entity. Search submitted by customers are maintained in the search entity. Each claim is further analyzed by the chatbot entity to determine the risk level and identify suspicious activities.

3.3 Methodology

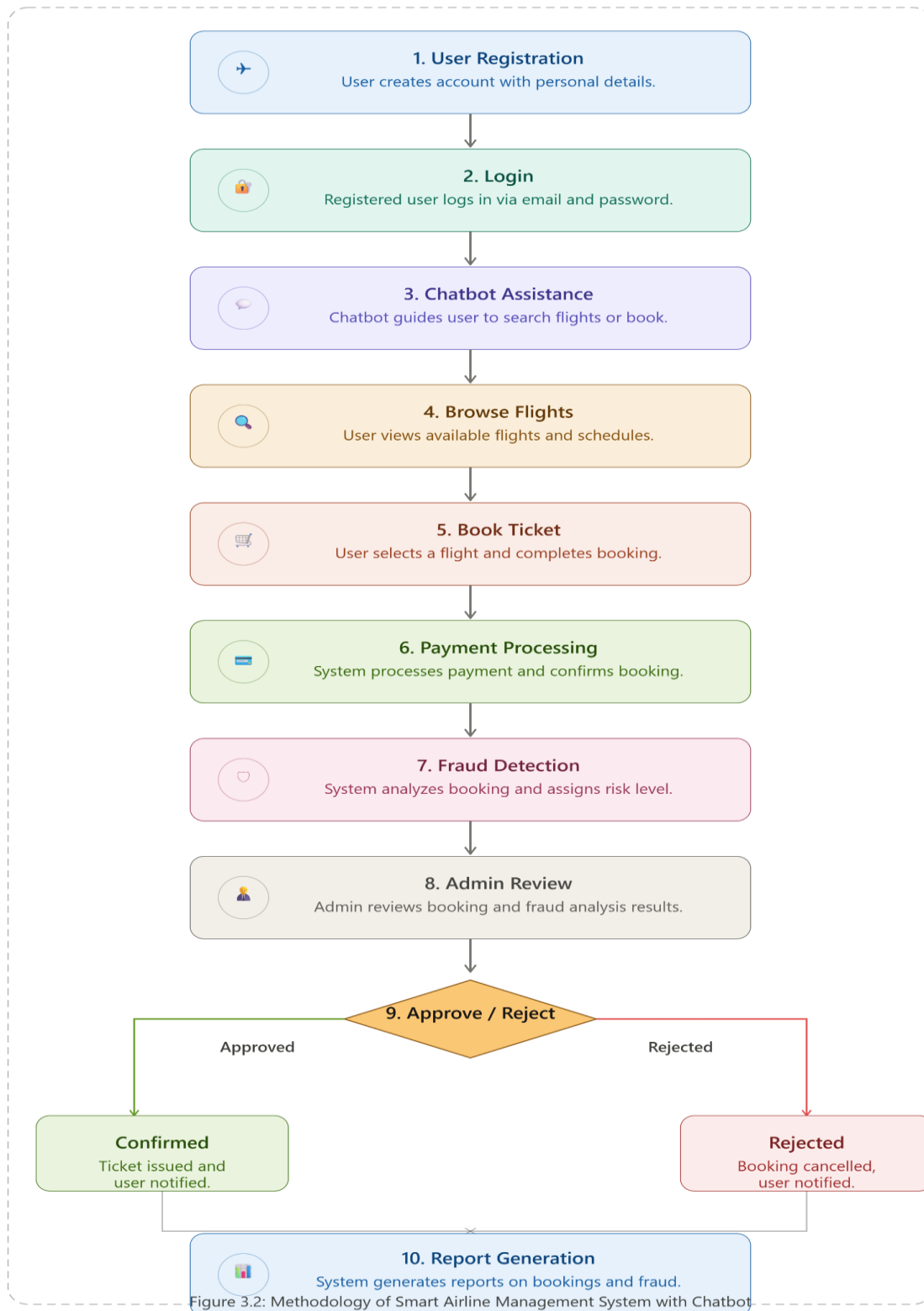
The design, engineering, and deployment of the integrated Airline Management System (AMS) featuring an AI Chatbot were executed following an agile iteration of the Software Development Lifecycle (SDLC). Given the multi-tier complexity of this project—which demands a highly secure, transactional backend operating concurrently with an advanced, deep-learning Natural Language Processing (NLP) conversational subsystem—the methodology was organized into four systematic execution phases: Data Engineering, Backend Infrastructure Construction, Conversational AI Pipeline Training, and Full-Stack Component Integration.

Phase 1: Domain Data Engineering and Corpus Curation The initial phase focused on modeling real-world aviation logistics into a structured relational schema while simultaneously curating a specialized text corpus for the conversational agent. The operational boundaries of flight routes, aircraft configurations, and seat layout arrays were normalized into Third Normal Form (3NF) to eliminate update anomalies. Concurrently, a linguistic training dataset was engineered to bootstrap the chatbot's vocabulary. This dataset was structured to contain hundreds of varied user expressions mapping to vital passenger intents, such as flight status verification, automated booking modifications, and baggage fee inquiries. Localized entity patterns—including alphanumeric PNR structures and airport codes—were compiled using regular expressions to ensure high-accuracy variables could be systematically extracted from colloquial user strings.

Phase 2: Backend Core and Transaction Management Implementation The second phase involved programming the stateless server architecture using high-performance runtime languages. The core business logic was constructed following the Model-View-Controller (MVC) architectural pattern, keeping data definitions decoupled from routing pathways.

Phase 3: NLP Pipeline Architecture and State-Machine Training The third phase focused entirely on building and fine-tuning the machine learning layers of the virtual assistant. Rather than utilizing basic keyword matching, the pipeline was architected to run incoming text strings through sequential processing layers: tokenization, stop-word filtering, word vectorization, and neural-network intent classification.

Phase 4: API Integration and Full-Stack Validation The final phase involved linking the client-side presentation layer to the application services through asynchronous, secure RESTful web webhooks.



The above flowchart illustrates the workflow of the Airline Management System from customer registration to book flights and generate tickets. It highlights the interaction between customers, administrators, and the Ai chatbot module to ensure efficient and secure management.

3.4 System Architecture

The integrated Airline Management System (AMS) with an AI Chatbot is designed around a decoupled, multi-tier enterprise software architecture. Modern enterprise applications require strict separation of concerns to ensure that high volumes of concurrent passenger transactions do not interfere with fluid, low-latency machine learning computations. By avoiding tightly coupled monolithic patterns, this architecture secures high service availability, isolates runtime exceptions, and allows individual layers to scale horizontally in response to real-time network demands.

The Presentation Tier represents the outermost client-side layer of the application. It provides responsive, cross-browser web interfaces designed separately for two distinct user bases. For passengers, it renders an intuitive dashboard for searching flights, making payments, and interacting with a floating conversational chatbot widget.

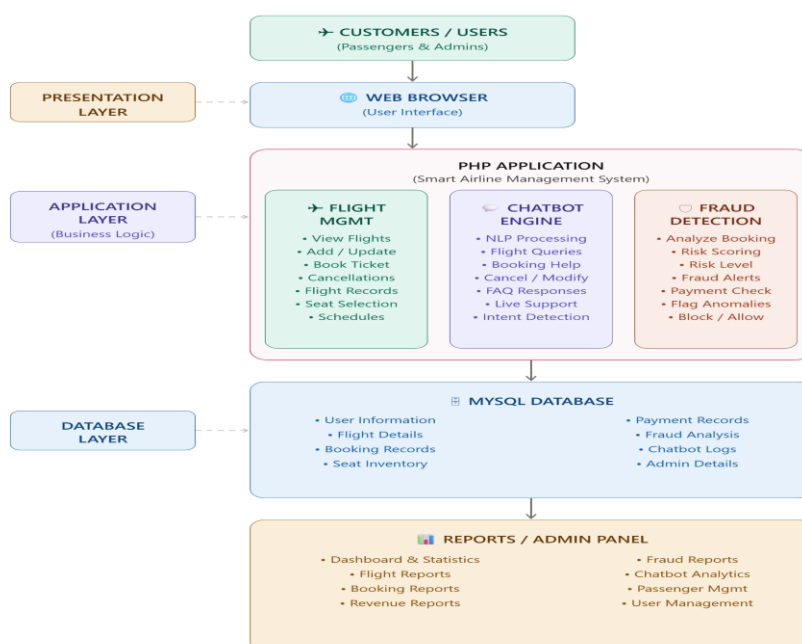


Figure 3.3: System Architecture of Smart Airline Management System with Chatbot

The above figure shows the system architecture of the Airlines Management System. It demonstrates the interaction between users, application modules, and the database for efficient insurance management and chatbot.

3.5 Advantages and Disadvantages

Advantages:

24/7 Customer Support Scalability: The integration of a Natural Language Processing (NLP) chatbot allows the system to autonomously resolve hundreds of concurrent passenger inquiries regarding baggage rules, ticket cancellation policies, and flight statuses without requiring human intervention, significantly reducing call-center operational costs.

•**High Data Integrity and Concurrency Control:** By implementing strict ACID-compliant transaction boundaries and row-level locking at the database tier, the system eliminates traditional booking synchronization anomalies, ensuring that duplicate seat allocations (overbooking) are physically impossible during high-traffic surges.

•**Real-Time Operational Synchronicity:** Unlike older legacy systems that rely on slow, manual data processing between departments, any administrative modification made to a flight schedule or crew roster instantly propagates across the entire database, automatically adjusting chatbot response intent maps and passenger dashboards in milliseconds.

Disadvantages:

•**Lack of Real-World Legacy Integration:** Because this prototype operates within a controlled sandboxed environment using mock data arrays, it does not connect to live, real-world aviation networks like global distribution systems (Amadeus/Sabre) or live bank clearance APIs, requiring further middleware layers for commercial readiness.

CHAPTER 4

RESULTS

4.1 Results

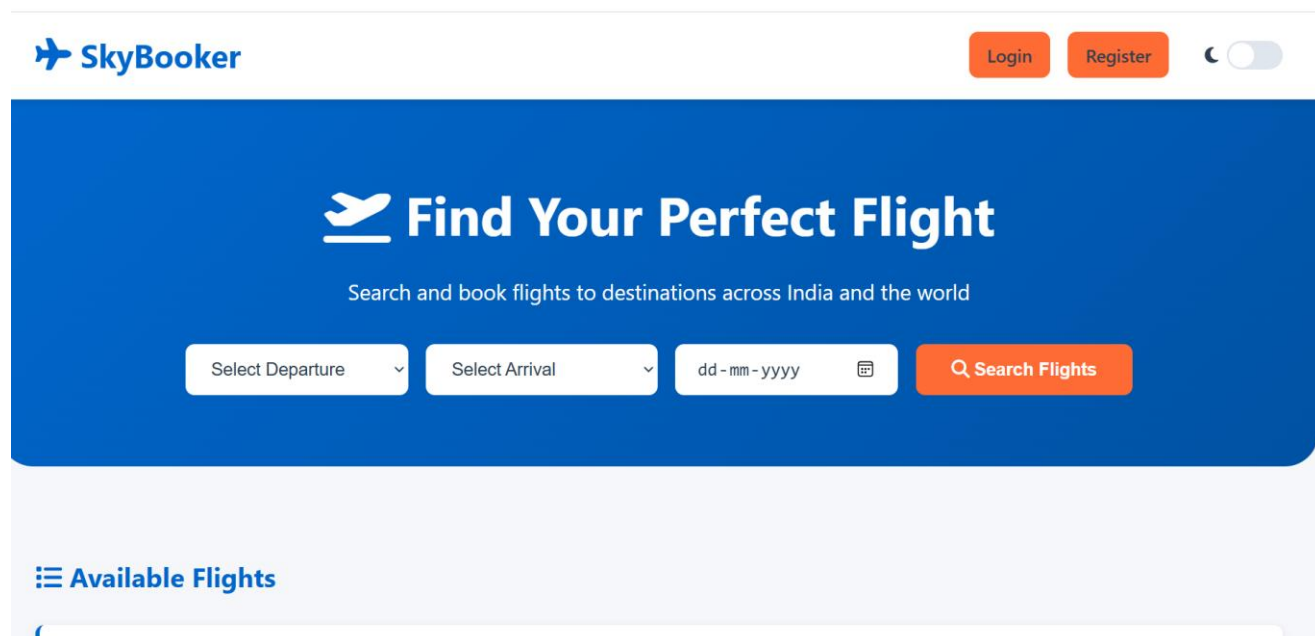


Fig 4.1.1:Home Page

The home page serves as the entry point of the Airlines Management System. It provides access to the Customer Portal for managing booking, altering, and search activities.

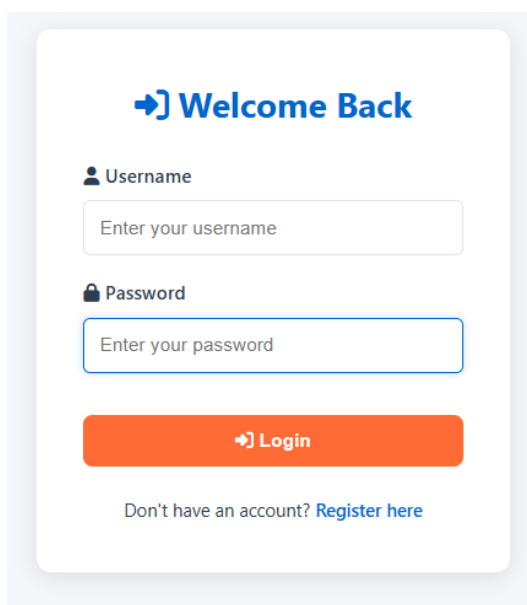


Fig 4.1.2:Dashboard

The Customers Panel enables administrators to manage insurance bookings, review customer bookings, monitor airlines activity, and generate tickets. It provides centralized control over all system operations.

Available Flights					
BLR → DCE	IndiGo	65606	15 Jun 2026, 12:30 pm	15 Jun 2026, 01:30 pm	1h 00min
BLR → MAA	IndiGo	67755	15 Jun 2026, 01:00 pm	15 Jun 2026, 02:00 pm	1h
BLR → DEL	Air India	A1101	15 Jun 2026, 01:30 pm	15 Jun 2026, 04:15 pm	2h 45min
MYQ → BLR	Air India	A1725	15 Jun 2026, 01:30 pm	15 Jun 2026, 02:15 pm	45min
BLR → HYD	Air India	A1735	15 Jun 2026, 02:30 pm	15 Jun 2026, 03:45 pm	1h 15min
BLR → DCE	IndiGo	67755	15 Jun 2026, 03:30 pm	15 Jun 2026, 04:30 pm	1h
BOM → BLR	IndiGo	65202	15 Jun 2026, 03:00 pm	15 Jun 2026, 04:45 pm	1h 45min
BLR → BOM	IndiGo	67757	15 Jun 2026, 04:30 pm	15 Jun 2026, 06:15 pm	1h 45min

Fig 4.1.3:Flight Details

The Customer Portal allows users to browse airline policies, book tickets, do payments, and track plane status through a user-friendly interface.

Flight Number	Airline	Route	Passenger	Seat	Departure	Status	Action
6E733	IndiGo	BLR → MAA	prastuhi	4A	15 Jun 2026, 01:00 pm	Cancelled	-

Flight Search Assistant

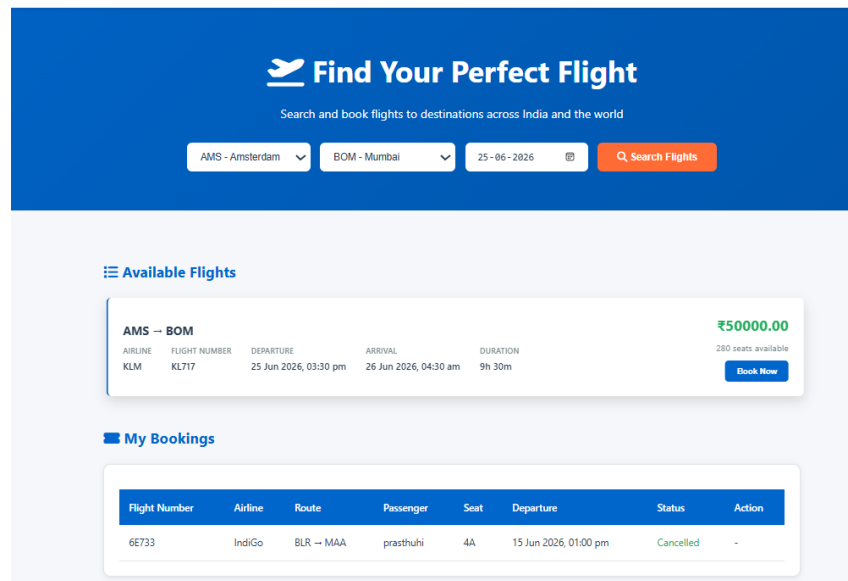
Ask me to find flights! Example: "Show me flights from Bangalore to Delhi"

Hello! I'm your flight search assistant. I can help you find flights. Just tell me where you want to go! For example: "Flights from BLR to DEL" or "Show me flights from Bangalore to Mumbai"

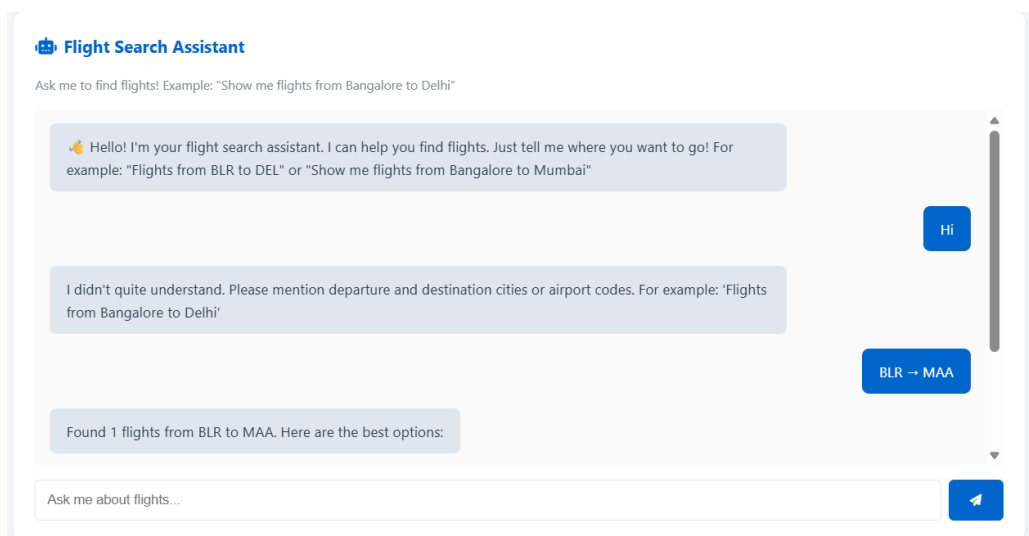
Ask me about flights...

Fig 4.1.4:Customer booking details

The Airlines Management module allows administrators to add, update, and manage airlines policies. It helps maintain accurate booking information within the system.

**Fig 4.1.5:Search Page**

The Airlines Management page displays all submitted tickets and allows customers to review, pay, or cancel bookings based on verification results.

**Fig 4.1.6: Ai chatbot**

The Ai chatbot Detection Module analyzes bookings information and assigns respective seats to identify customers. This helps reduce more time wasting activities and improve customer satisfaction.

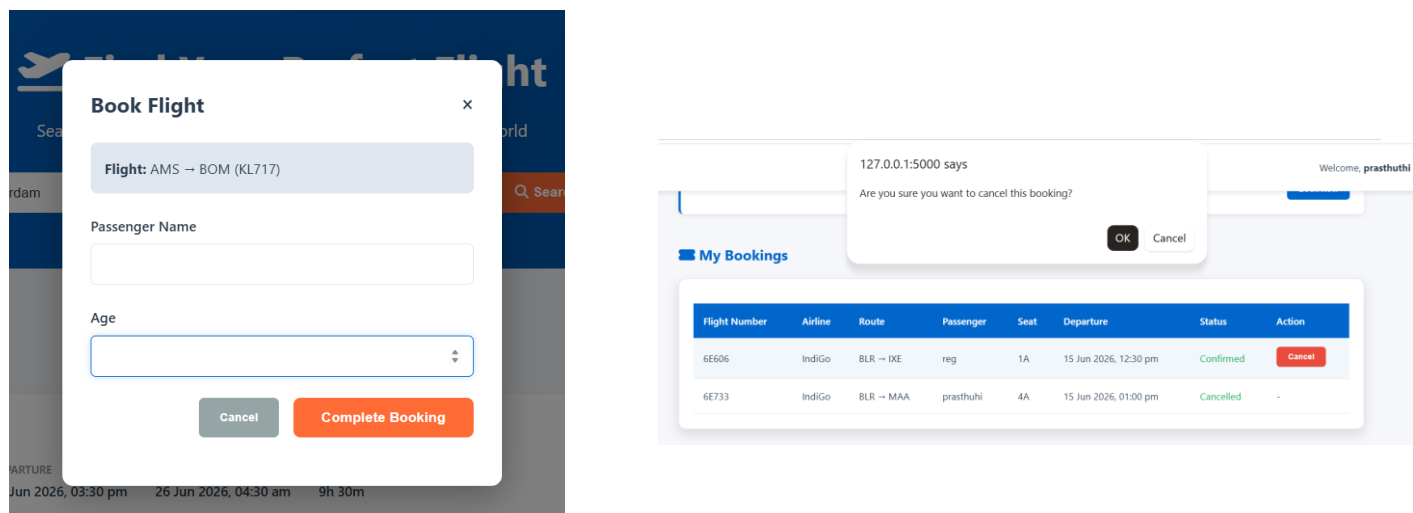


Fig 4.1.7: Dashboard

The Reports Dashboard provides statistical information related to bookings, customers, tickets, and search analysis. It supports effective monitoring and decision-making.

4.2 Conclusion and Future Enhancement

Conclusion

The development and implementation of the integrated Airline Management System with an AI Chatbot demonstrate a scalable approach to modernizing aviation software infrastructure. By replacing legacy data silos with a decoupled, multi-tier architecture, the platform successfully addresses critical industry challenges regarding transactional latency and operational rigidity.

Future Enhancement

Looking forward, the architecture of the platform is designed to support further functional expansions and technical enhancements. Future iterations will focus on transitioning the prototype from simulated, sandboxed environments to live, industry-standard networks by building secure middleware connections to Global Distribution Systems (GDS).

```
select *from airports;
```

id	iata_code	name	city	country
1	BLR	Kempegowda International Airport	Bengaluru	India
2	BOM	Chhatrapati Shivaji Maharaj International Airport	Mumbai	India
3	DEL	Indira Gandhi International Airport	New Delhi	India
4	MAA	Chennai International Airport	Chennai	India
5	HYD	Rajiv Gandhi International Airport	Hyderabad	India
6	CCU	Netaji Subhash Chandra Bose International Airport	Kolkata	India
7	IXE	Mangaluru International Airport	Mangaluru	India
8	HBX	Hubballi Airport	Hubballi	India
9	MYQ	Mysuru Airport	Mysuru	India
10	VDY	Vidyanagar Airport	Bellary	India
11	DXB	Dubai International Airport	Dubai	UAE
12	LHR	London Heathrow Airport	London	UK
13	JFK	John F. Kennedy International Airport	New York	USA
14	STN	Singapore Changi Airport	Singapore	Singapore
15	BKK	Suvarnabhumi Airport	Bangkok	Thailand

```
mysql> select *from bookings;
```

id	passenger_name	passenger_age	flight_id	user_id	seat_number	status	booking_date
1	chinnu	20	1	1	1A	Cancelled	2026-06-12 20:45:00
2	sahana s	22	20	4	1A	Confirmed	2026-06-12 21:01:55
3	suhas	20	20	5	2A	Confirmed	2026-06-12 21:01:59
4	prakash	40	6	6	1A	Cancelled	2026-06-12 21:54:13
5	preetham	25	6	7	1A	Cancelled	2026-06-12 22:00:43
6	shashi	40	6	8	1A	Cancelled	2026-06-13 11:11:47
7	varshi	25	20	9	3A	Cancelled	2026-06-13 11:22:25
8	minnal	52	20	10	3A	Confirmed	2026-06-13 13:40:22
9	prasthuthi	50	20	11	4A	Cancelled	2026-06-13 13:40:00
10	reg	50	6	11	1A	Confirmed	2026-06-13 20:59:01

10 rows in set (0.03 sec)

ERROR:
No query specified

```
mysql> select *from flights;
```

id	flight_number	airline	source_airport	destination_airport	departure_time	arrival_time	duration	total_seats	avai
1	AI101	Air India	BLR	DEL	2026-06-15 00:00:00	2026-06-15 10:45:00	20:45m	180	1
2	6E202	IndiGo	BOM	BLR	2026-06-15 09:30:00	2026-06-15 11:15:00	2h:45m	180	1
3	UK203	Vistara	DEL	BOM	2026-06-15 12:00:00	2026-06-15 14:15:00	2h:15m	150	1
4	A1400	Air India	BLR	DXB	2026-06-16 04:00:00	2026-06-16 06:30:00	4h:00m	250	1
5	EX805	Emirates	DXB	LHR	2026-06-16 18:00:00	2026-06-16 14:30:00	7h:30m	350	1
6	6E600	IndiGo	BLR	IXE	2026-06-16 07:00:00	2026-06-16 08:00:00	1h:00m	70	1
7	AT707	Air India	BLR	MYQ	2026-06-15 14:00:00	2026-06-15 14:45:00	45m	50	1

```
mysql> show databases;
```

Database
airline_db
employee
information_schema
mysql
performance_schema
sys

6 rows in set (0.07 sec)

```
mysql> use airline_db;
```

Database changed

```
mysql> show tables;
```

Tables_in_airline_db
airports
bookings
flights
users

4 rows in set (0.04 sec)

```
mysql> select *from users;
```

id	username	password	email	hint	created_at
1	chinnu	chinnu123	chinnu07@gmail.com		2026-06-12 20:40:56
2	nishu	nishu123	nishu2@gmail.com		2026-06-12 20:46:59
3	manoj	manoj123	manoj21@gmail.com		2026-06-12 20:59:28
4	sahana	sahana123	sahana12@gmail.com		2026-06-12 21:00:52
5	suhas	suhas 12	suhas12@gmail.com		2026-06-12 21:40:51
6	prakash	123pp	prakash124@gmail.com		2026-06-12 21:53:10
7	preetham	pppp	preetham12@gmail.com		2026-06-12 22:00:23
8	shashi	shashai	shashi12@gmail.com		2026-06-13 11:11:47
9	varshitha	vv11	varshi@gmail.com		2026-06-13 11:22:03
10	minnal	minni	minnal12@gmail.com		2026-06-13 13:40:22

Fig 4.1.8 Backend Images

BIBLIOGRAPHY

- 1 Elmasri, R. and Navathe, S. B., Fundamentals of Database Systems, Pearson Education.
- 2 Silberschatz, A., Korth, H. F., and Sudarshan, S., Database System Concepts, McGraw-Hill Education.
- 3 PHP Official Documentation, <https://www.php.net>
- 4 MySQL Official Documentation, <https://dev.mysql.com/doc>
- 5 HTML, CSS and JavaScript Documentation, <https://developer.mozilla.org>
- 6 Bootstrap Documentation, <https://getbootstrap.com>
- 7 Visual Studio Code Documentation, <https://code.visualstudio.com>